ACME: Adaptive Cognitive Memory Engine

Externalizing Belief, Memory, and Learning from LLM Weights

arXiv preprint draft v1.0 — June 2026

Author: Mohamed Kamil Bourouiba
Code: https://github.com/KamilBourouiba/ACME (tag `v0.1.1-arxiv`, commit `main`)
Project site: https://kamilbourouiba.github.io/ACME/
Data: Production benchmark exports via `GET /api/v1/benchmark/export` (persisted in Postgres `benchmark_runs`)

Abstract

Large language models lack durable episodic memory, explicit belief states, structured forgetting, and mechanisms to learn from failure. We present **ACME** (Adaptive Cognitive Memory Engine), an external cognitive substrate that treats the LLM as a language processor while delegating persistence, confidence tracking, contradiction handling, and self-correction to specialized engines. ACME operationalizes: *when does a collection of experiences deserve to become a belief?* We contribute (1) a promotion/demotion lifecycle with Cognitive Reliability Score (CRS), (2) hybrid graph + pgvector retrieval with contrarian verification, (3) **MemoryBench v3** — thirteen sandbox-isolated scenarios with RAG, MemGPT, and LangGraph baselines scored by an LLM judge. On Azure OpenAI GPT-4.1 with `text-embedding-3-small` (256D) and Postgres Flexible + pgvector, ACME achieves overall **0.925** vs **0.487** for vector-RAG (MemoryBench v3, 24 June 2026 production run, job `3b31e5e3`), with full feedback and belief-quality metrics unavailable to baselines.

---

1. Introduction

Retrieval-augmented generation (RAG) augments LLMs with document search but does not maintain explicit epistemic states, handle contradictory evidence, or learn from prediction failures. Agent memory frameworks (MemGPT, LangGraph) improve session persistence but lack auditable belief lifecycles and structured feedback loops.

ACME externalizes cognition into specialized engines:

- **Episodic store** (PostgreSQL + pgvector) and **semantic graph** (Neo4j, tenant-scoped)
- **Belief engine** — observation -> hypothesis -> belief -> challenged -> deprecated -> archived
- **Contrarian verification** on high-confidence answers
- **Compression, forgetting, and autonomous learning** with prediction validation
- **MemoryBench v3** for reproducible evaluation with CI gates

---

2. System Design

Ingestion: LLM + deterministic extraction -> Neo4j entities/relations + episodic embeddings.

Query: Hybrid retrieval (graph neighborhood + pgvector cosine) -> LLM reasoning -> optional contrarian pass.

Feedback: Outcome signals update beliefs, log failures, trigger consolidation (Azure cron, 6h).

CRS = 40% prediction success + 20% temporal stability + 20% contradiction resistance + 20% source diversity.

Causal edge types: `observed_with`, `precedes`, `correlates`, `causes`, `disproves`.

Multi-tenancy: Postgres rows and Neo4j nodes keyed by `tenant_id` (header `X-Tenant-ID`).

Ablation toggles: `ABLATION_DISABLE_CONTRARIAN`, `ABLATION_DISABLE_BELIEF_SYNC`, `ABLATION_DISABLE_VECTOR` (see Section 5).

---

3. MemoryBench v3

Four metrics, LLM-as-judge (semantic; keyword overlap reported separately):

Metric	Description
Retention	Concept coverage (synonyms accepted)
Groundedness	Answer supported by ingested episodes
Feedback correction	Belief adjustment after contradictions
Belief quality	Mean CRS across tracked beliefs

Overall = average of all four. Baselines score 0 on feedback/belief (N/A), capping baseline overall near 0.48-0.49.

Thirteen scenarios: retention, contradiction, multi-source conflict, error injection, long-term retention, hallucination resistance, feedback adjustment, adversarial noise, long-horizon noise, tenant isolation, healthcare domain transfer, multi-session recall, prediction-outcome loop.

Isolation: Each scenario deletes prior benchmark-tagged Postgres rows and Neo4j subgraph before run (`acme/evaluation/sandbox.py`).

Baselines: Minimal reproducible implementations aligned with Lewis et al. (RAG), Packer et al. (MemGPT), LangGraph-style state graphs — see `docs/BASELINES.md`.

---

4. Experimental Setup

Component	Configuration
LLM	Azure OpenAI GPT-4.1
Embeddings	`text-embedding-3-small`, 256 dimensions
Database	Azure Postgres Flexible (pgvector), francecentral
Graph	Neo4j 5.x on Azure Container Apps
Judge	Same GPT-4.1 deployment (`evaluate_answer_quality`)
Reproducibility	`POST /api/v1/benchmark/memorybench`, `POST /api/v1/benchmark/compare/async`

---

5. Results (Azure production, 24 June 2026, 13 scenarios, isolated)

Run metadata: API revision `acme-api--membenchfix`; compare job `3b31e5e3-72b1-4ce3-b6c2-848cf94e4128`; duration 725 s; zero scenario failures.

System	Retention	Groundedness	Feedback	Belief Q.	Overall
ACME	**1.000**	**1.000**	**1.000**	**0.700**	**0.925**

System	Retention	Groundedness	Feedback	Belief Q.	Overall
RAG baseline	0.969	0.977	N/A	N/A	0.487
MemGPT baseline	0.977	0.969	N/A	N/A	0.487
LangGraph baseline	0.900	0.977	N/A	N/A	0.469

Baselines exclude feedback/belief in overall (not applicable). Scores rounded to three decimals from persisted `benchmark_runs` export.

Key finding: ACME gains **+0.44** overall vs RAG primarily from feedback correction and belief quality — dimensions absent from all baselines. Retention and groundedness are competitive across systems (all ≥ 0.90 on applicable metrics).

Model sensitivity (GPT-4.1-mini): Using the same pipeline with Azure OpenAI **GPT-4.1-mini** as extractor, reasoner, and judge (job `107d02c0`, 639 s), ACME retains a **+0.39** overall advantage vs RAG (0.858 vs 0.473). Retention and groundedness drop versus GPT-4.1, but feedback correction stays at 1.000 and belief quality at 0.700 — confirming that cognitive layers compensate when the base model is weaker.

System	Retention	Groundedness	Feedback	Belief Q.	Overall
ACME	**0.892**	**0.838**	**1.000**	**0.700**	**0.858**
RAG baseline	0.908	0.985	N/A	N/A	0.473
MemGPT baseline	0.915	1.000	N/A	N/A	0.479
LangGraph baseline	0.854	0.862	N/A	N/A	0.429

Ablation (design): Disabling contrarian checks, belief sync, or vector retrieval is supported via environment flags; unit gates verify toggles (`scripts/run_ablation_gate.py`). Full ablation sweeps on live LLM runs are left to future work due to cost.

---

6. Limitations

- LLM judge and extractor introduce variance; we report sandbox-isolated runs with persisted `benchmark_runs`.
- Scenarios emphasize SaaS churn/latency plus one healthcare transfer set; broader domains needed.
- Baselines are reference implementations, not full upstream MemGPT/LangGraph codebases.
- Production API requires API key for benchmark endpoints.

---

7. Conclusion

ACME shows that externalizing belief and learning from LLM weights yields stronger cognitive memory than vector RAG or lightweight agent-memory baselines on MemoryBench v3, especially for feedback-driven correction and auditable belief quality.

References

1. Lewis, P. et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. 2020.
2. Packer, C. et al. MemGPT: Towards LLMs as Operating Systems. 2023.
3. Anderson, J. R. ACT-R cognitive architecture.
4. ACME repository: https://github.com/KamilBourouiba/ACME

Appendix A — Reproduction

```
git clone https://github.com/KamilBourouiba/ACME.git && cd ACME
pip install -e ".[dev]"
pytest tests/test_benchmark_gate.py tests/test_memorybench.py -q
# Production (requires API key):
./azure/configure-premium-ingress.sh
./scripts/run_prod_benchmark.sh
```

Appendix B — Data Availability

Benchmark payloads stored in PostgreSQL table `benchmark_runs` (columns: `overall_score`, `retention_score`, `belief_quality_score`, `payload JSONB`, `created_at`). Export via `GET /api/v1/benchmark/export`.